feature velocity (different parts of the application can be written and deployed independently), partial availability (while certain parts of the application might not be available to the end user, the overall application might still be working), as well as enabling us to use the best-fitting programming languages and data stores for a certain functionality (e.g., JavaScript for a frontend or an in-memory key–value store, like Redis, for a shopping basket).

> **NOTE** We previously covered feature velocity as an advantage of microservices-based architectures. Diving deeper here, the cause of increased feature velocity is that smaller units make for an increase in the deployment velocity. In other words, if your deployment units are smaller, they are easier to test, upgrade, and fix. It's worth noting that some enterprises have achieved increased development velocity even with what would be considered a monolith, when applying the "small units" idea to strictly decoupled modules in the monolith.

On the flipside, microservices are, indeed, distributed systems, and as such, they are more complex to operate than monolithic apps. Observability is not optional anymore. Part of the problem is determining where something happened. You don't have to be eBay, Netflix, or Uber, dealing with hundreds of microservices, to face this problem. Even if you "only" have a couple dozen services making up your cloud-native app, figuring out exactly which of the services along the request path caused an error or contributed to a slower-than-usual response can be tricky.

So how can you get an overview of how all your services play together and spot services that don't work as expected? Perhaps, you could drill down into a specific service to examine how it is performing.

> **NOTE** One challenge with distributed systems is that (potentially) services are running on different machines. The logs of these services may come with timestamps that are not 100% aligned (http://mng.bz/Bmp0), which is tricky to handle, since it ruins log ordering across services, hence, breaking the cause–effect analysis. Even when ingesting the logs into a central place, the dynamics of the execution would usually need to be manually restored, based on tags or labels. So while it's possible to use logs alone in the context of microservices, it is not the most effective use of your time and may require more skill on the part of the analyst than the solution we discuss in this chapter.

In this chapter, we have a closer look at a solution to the aforementioned challenge: distributed tracing. We discussed the topic on a high level in chapter 2, but now, it's time to dive deeper. I will first provide an overall introduction to this relatively new concept in the observability space, and then we will see distributed tracing in action in the context of a concrete use case. Finally, we will close out the chapter with practical considerations around selecting and using distributed tracing solutions.